Universidad de La Laguna

Grado en Ingeniería Informática

Computabilidad y Algoritmia

Práctica 5

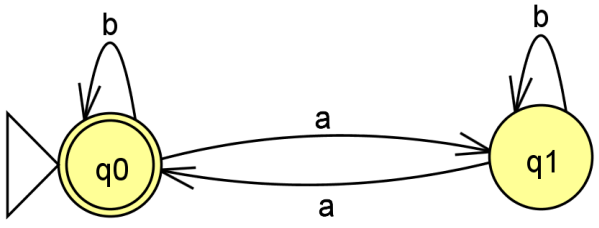
Diseño y simulación de autómatas finitos en JFLAP

Luis Eduardo Martín Hernández
alu0101631134@ull.edu.es

3 Diseño de DFAs:

1. **Diseñar un autómata finito determinista que reconozca cadenas sobre el alfabeto Σ = {a, b} con número de “a’s” par.**

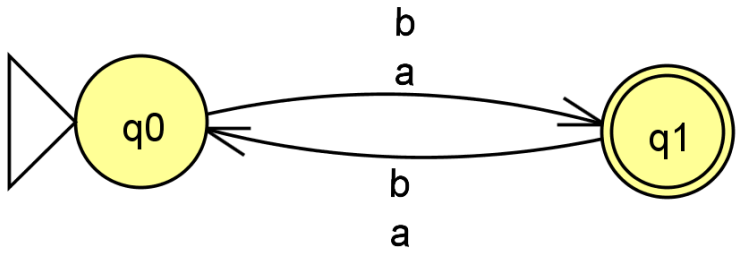
Un estado para las cadenas de longitud par (estado de aceptación) y otro estado para las cadenas de longitud impar únicamente conectados si la entrada es una “a”.



	Accept
aa	Accept
aba	Accept
baabaabb	Accept
b	Accept
bab	Reject
a	Reject
ba	Reject
aaab	Reject
aaaaaaaaabbaa	Reject

2. **Diseñar un autómata finito determinista que reconozca cadenas sobre el alfabeto Σ = {a, b} con longitud impar**

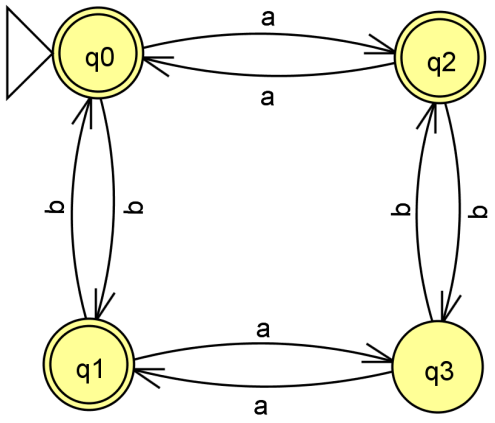
Un estado para las cadenas de longitud par y otro estado para las cadenas de longitud impar (estado de aceptación).



b	Accept
a	Accept
abababa	Accept
aaaab	Accept
baaba	Accept
	Reject
aa	Reject
babb	Reject
aaba	Reject
aaaaaaaaabba	Reject

3. **Diseñar un autómata finito determinista que reconozca cadenas sobre el alfabeto Σ = {a, b} con número de "a's" par o longitud impar.**

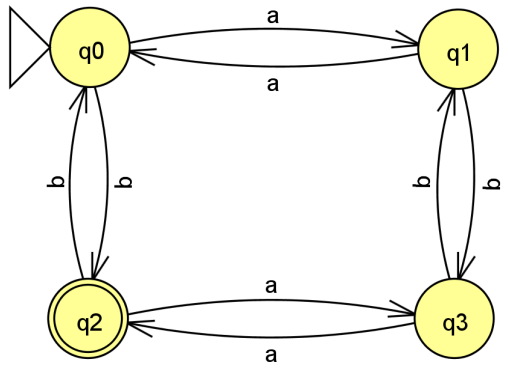
Son necesarios 4 estados para todas las combinaciones entre longitud par o impar y número de "a's" par o impar. Los estados de aceptación son aquellos en los que se cumple al menos una de estas dos condiciones.



	Accept
aa	Accept
abbaa	Accept
aabb	Accept
a	Accept
ab	Reject
bbab	Reject
ba	Reject
bbbaaa	Reject
bababa	Reject

4. **Diseñar un autómata finito determinista que reconozca cadenas sobre el alfabeto Σ = {a, b} con número de “a’s” par y longitud impar.**

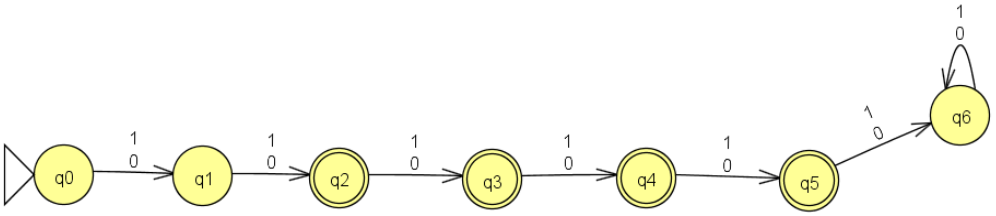
Exactamente igual que el anterior pero sólo será estado de aceptación aquel que cumpla ambas condiciones.



aaaab	Accept
aab	Accept
abbab	Accept
b	Accept
baa	Accept
bba	Reject
	Reject
a	Reject
abbbbbb	Reject
babbbaab	Reject

5. **Diseñar un autómata finito determinista que reconozca cadenas w sobre el alfabeto {0, 1} tales que 2 ≤ |w| ≤ 5.**

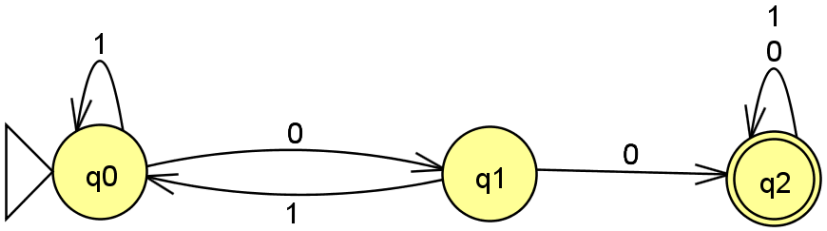
Seis estados para cada longitud posible hasta 5 de los cuales son aceptación los pertenecientes al intervalo (ambos extremos incluidos) y un último estado de absorción o muerte por si la longitud de la cadena es mayor.



00	Accept
011	Accept
1100	Accept
10101	Accept
111	Accept
	Reject
0	Reject
1	Reject
010100101	Reject
101001	Reject

6. **Diseñar un autómata finito determinista que reconozca cadenas sobre el alfabeto {0, 1} que tengan como mínimo dos ceros consecutivos.**

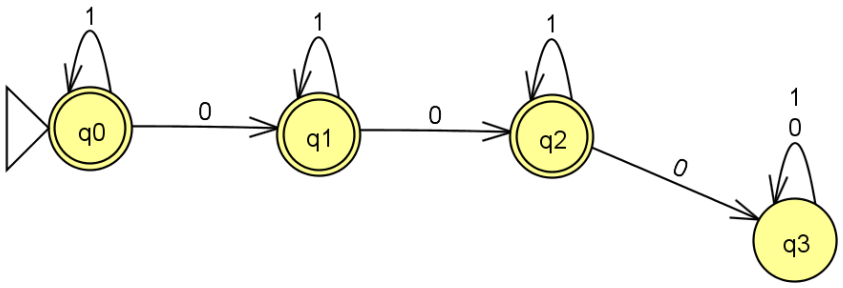
Son necesarios tres estados, un primer estado cuando no se ha detectado ningún 0, un segundo estado para cuando se ha detectado un 0 y un tercer estado en el que ya se han detectado los dos 0 consecutivos (estado de aceptación).



001	Accept
11111110111100	Accept
10011111101	Accept
1110010	Accept
00	Accept
	Reject
1	Reject
01	Reject
010	Reject
1111010101	Reject

7. **Diseñar un autómata finito determinista que reconozca cadenas sobre el alfabeto {0, 1} que tengan como máximo dos ceros.**

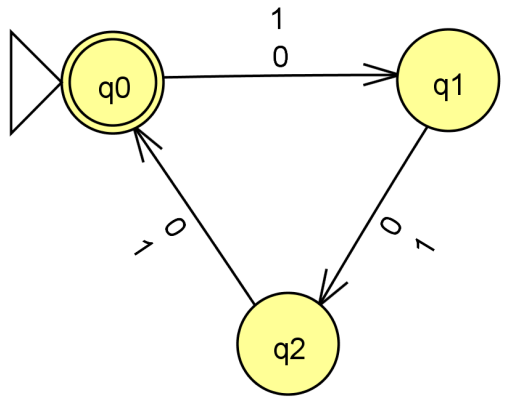
Son necesarios 4 estados, los tres primeros se encargan de comprobar que han recibido como máximo dos ceros, en cualquier posición de la cadena (los tres estados de aceptación) y el último es un estado de absorción o muerte por si recibe un 0 adicional.



	Accept
1111111011110	Accept
1001111111	Accept
1110	Accept
0	Accept
10001	Reject
0010	Reject
000000111111	Reject
0101011	Reject
111101010	Reject

8. **Diseñar un autómata finito determinista que reconozca cadenas sobre el alfabeto {0, 1} con longitud múltiplo de 3.**

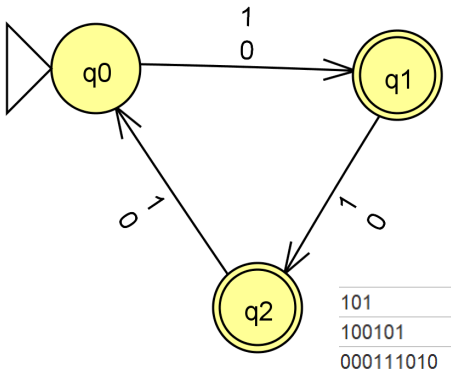
Son necesarios tres estados con el estado de aceptación en el estado inicial (0 es múltiplo de 3) y los otros dos estados corresponden con los otros dos restos restantes al dividir un número entre 3.



	Accept
101	Accept
100101	Accept
000111010	Accept
011	Accept
10001	Reject
0010	Reject
0000001111111	Reject
0101011	Reject
1111010100	Reject

9. **Diseñar un autómata finito determinista que reconozca cadenas sobre el alfabeto {0, 1} con una longitud que no sea múltiplo de 3.**

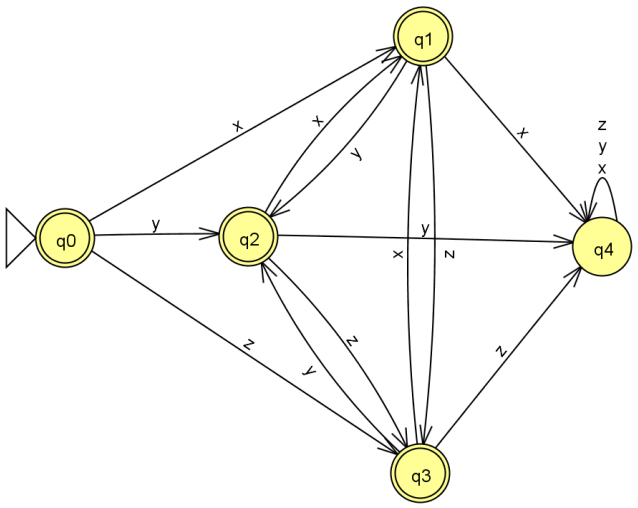
Como se trata de un lenguaje complementario con el del ejercicio anterior solo debemos intercambiar los estados de aceptación con los de no aceptación.



	Reject
101	Reject
100101	Reject
000111010	Reject
011	Reject
10001	Accept
0010	Accept
0000001111111	Accept
0101011	Accept
1111010100	Accept

10. Diseñar un autómata finito determinista que reconozca cadenas sobre el alfabeto {x, y, z} que no contengan dos símbolos iguales consecutivos.

Son necesarios 4 estados de aceptación, el primero (estado inicial) para cuando no se ha recibido ningún símbolo y los otros tres, cada uno asignado con un símbolo. El quinto estado es un estado de absorción o de muerte en caso de que se reciban dos símbolos iguales seguidos.

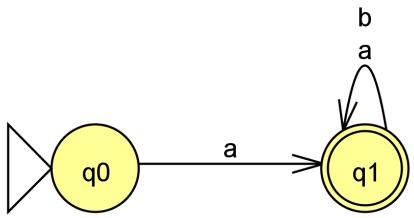


	Accept
x	Accept
y	Accept
z	Accept
xyzxyx	Accept
xxy	Reject
xyyz	Reject
yy	Reject
zz	Reject
yyxxzz	Reject

4 Diseño de NFAs:

1. **Diseñar un autómata finito no determinista que reconozca cadenas sobre el alfabeto Σ = {a, b} que empiecen por “a”. A partir del NFA diseñado, obtenga un DFA mínimo equivalente.**

Son necesarios dos estados, un estado inicial que solo da la opción de recibir como entrada una “a” de esta manera accedería al segundo estado (estado de aceptación).



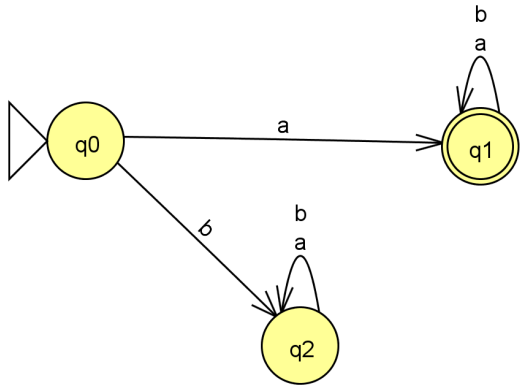
abbaa	Accept
a	Accept
ab	Accept
aa	Accept
aaaaa	Accept
	Reject
b	Reject
ba	Reject
bb	Reject
bbbaa	Reject

A continuación aplicaré el algoritmo de construcción de subconjuntos a un NFA sin ε − transiciones

$Q' = 2^Q = \{Ø, \{q_0\}, \{q_1\}, \{q_0, q_1\}\}$
$δ'(Ø, a) = δ'(Ø, b) = Ø$
$δ'(\{q_0\}, a) = δ(q_0, a) = \{q_1\}$
$δ'(\{q_0\}, b) = δ(q_0, b) = Ø$
$δ'(\{q_1\}, a) = δ(q_1, a) = \{q_1\}$
$δ'(\{q_1\}, b) = δ(q_1, b) = \{q_1\}$

(Ya que no sona accesibles desde el estado inicial, no es necesario comprobar el conjunto de estados $\{q_0, q_1\}$)

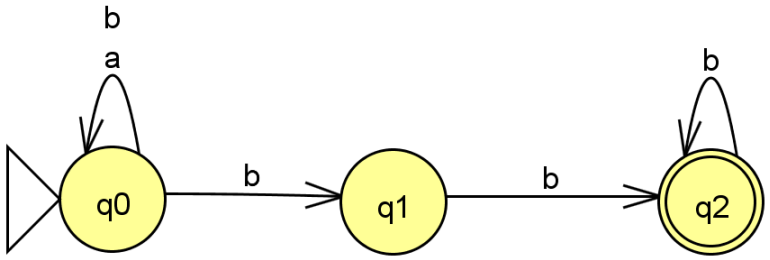
DFA resultante:



Aunque parece que al obviar el conjunto $\{q_0, q_1\}$ el autómata ya está minimizado vamos a comprobarlo aplicando el algoritmo de minimización de estados.
$Π = \{\{0, 2\}, \{1\}\}$
$Π = \{\{0\}, \{2\}, \{1\}\}$
Efectivamente, el autómata ya era mínimo.

2. **Diseñar un autómata finito no determinista que reconozca cadenas sobre el alfabeto Σ = {a, b} que terminen en “bb”. A partir del NFA diseñado, obtenga un DFA mínimo equivalente.**

Son necesarios 3 estados, un primer estado inicial en el que no afecte la entrada y los otros dos estados encargados de comprobar si al menos los dos últimos símbolos de la cadena son una “b” siendo únicamente el último un estado de aceptación.

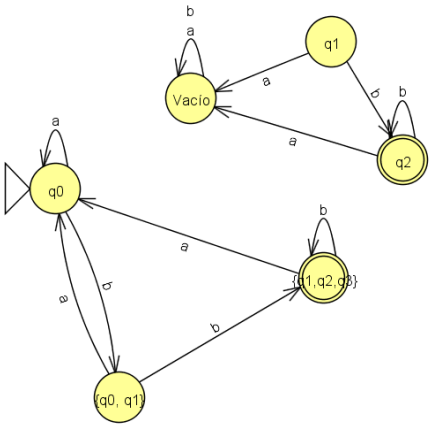


abababbb	Accept
bb	Accept
bbbbbb	Accept
aabbaabbb	Accept
abb	Accept
	Reject
ab	Reject
a	Reject
baab	Reject
aaaabab	Reject

A continuación aplicaré el algoritmo de construcción de subconjuntos a un NFA sin ε − transiciones

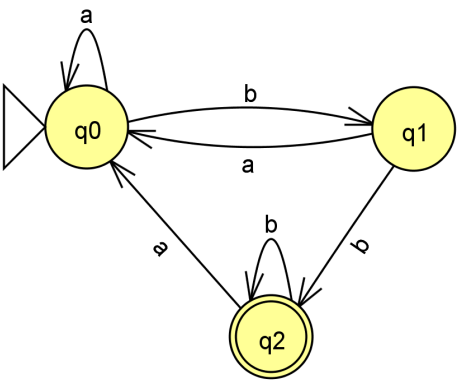
$Q' = 2^Q = \{\varnothing, \{q_0\}, \{q_1\}, \{q_2\}, \{q_0, q_1\}, \{q_0, q_2\}, \{q_1, q_2\}, \{q_0, q_1, q_2\}\}$
$\delta'(\varnothing, a) = \varnothing$, $\delta'(\varnothing, b) = \varnothing$
$\delta'(\{q_0\}, a) = \delta(q_0, a) = \{q_0\}$
$\delta'(\{q_0\}, b) = \delta(q_0, b) = \{q_0, q_1\}$
$\delta'(\{q_1\}, a) = \varnothing$
$\delta'(\{q_1\}, b) = \delta(q_1, b) = \{q_2\}$
$\delta'(\{q_2\}, a) = \varnothing$
$\delta'(\{q_2\}, b) = \delta(q_2, b) = \{q_2\}$
$\delta'(\{q_0, q_1\}, a) = \delta(q_0, a) \cup \delta(q_1, a) = \{q_0\} \cup \varnothing = \{q_0\}$
$\delta'(\{q_0, q_1\}, b) = \delta(q_0, b) \cup \delta(q_1, b) = \{q_0, q_1\} \cup \{q_2\} = \{q_0, q_1, q_2\}$
$\delta'(\{q_0, q_2\}, a) = \delta(q_0, a) \cup \delta(q_2, a) = \{q_0\} \cup \varnothing = \{q_0\}$
$\delta'(\{q_0, q_2\}, b) = \delta(q_0, b) \cup \delta(q_2, b) = \{q_0, q_1\} \cup \{q_2\} = \{q_0, q_1, q_2\}$
$\delta'(\{q_1, q_2\}, a) = \varnothing \cup \varnothing = \varnothing$
$\delta'(\{q_1, q_2\}, b) = \delta(q_1, b) \cup \delta(q_2, b) = \{q_2\} \cup \{q_2\} = \{q_2\}$
$\delta'(\{q_0, q_1, q_2\}, a) = \delta(q_0, a) \cup \delta(q_1, a) \cup \delta(q_2, a) = \{q_0\} \cup \varnothing \cup \varnothing = \{q_0\}$
$\delta'(\{q_0, q_1, q_2\}, b) = \delta(q_0, b) \cup \delta(q_1, b) \cup \delta(q_2, b) = \{q_0, q_1\} \cup \{q_2\} \cup \{q_2\} = \{q_0, q_1, q_2\}$

Ya que los conjuntos de estados {q0, q2} y {q1, q2} son inaccesibles, se pueden obviar.



Además, al dibujar el autómata se puede apreciar que tanto el conjunto ∅ como {q1} y {q2} también son inalcanzables desde el estado inicial así que también podemos obviarlos.

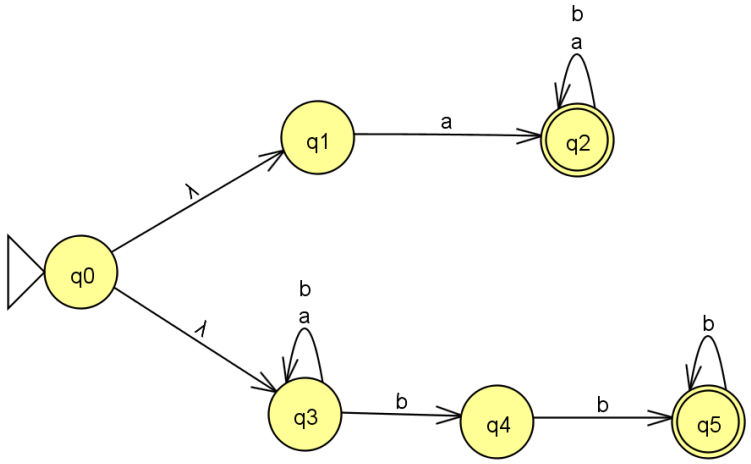
DFA resultante:



Debido a que la cantidad de estados del DFA es igual que la cantidad de estados del NFA sin ε-transiciones sabemos que este autómata es mínimo.

3. **Diseñar un autómata finito no determinista que reconozca cadenas sobre el alfabeto Σ = {a, b} que empiecen por “a” o terminen en “bb”. A partir del NFA diseñado, obtenga un DFA mínimo equivalente.**

Uniendo los dos NFAs anteriores a un estado inicial a través de dos ε-transiciones para la disyunción consigo un NFA que atiende a la disyunción de los dos casos.



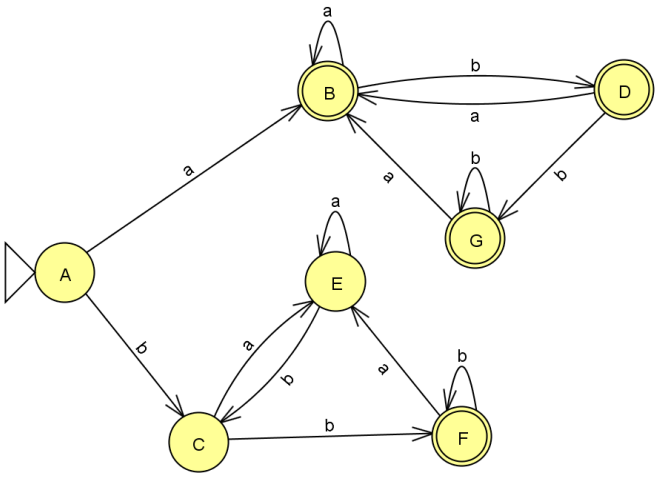
a	Accept
babb	Accept
aab	Accept
abbbabab	Accept
bbabb	Accept
	Reject
ba	Reject
bbaab	Reject
baa	Reject
bbbaaaab	Reject

A continuación aplicaré el algoritmo de construcción de subconjuntos.

q0’ = ε − clausura (q0) = {q_0, q_1, q_3}
Como se trata de una disyunción a partir de ε-transiciones podemos simplificarlo diciendo que A es el estado inicial y quitar a q_0 de su conjunto.
A = {q1, q3}
δ(A, a) = {q_2, q_3}
ε − clausura ({q_2, q_3}) = {q_2, q_3} = B | δ’(A, a) = B
δ(A, b) = {q_3, q_4}
ε − clausura({q_3, q_4}) = {q_3, q_4} = C | δ’(A, b) = C
δ(B, a) = {q_2, q_3}
ε − clausura ({q_2, q_3}) = B | δ’(B, a) = B
δ(B, b) = {q_2, q_3, q_4}
ε − clausura({q_2, q_3, q_4}) = {q_2, q_3, q_4} = D | δ’(B, b) = D
δ(C, a) = {q_3}
ε − clausura({q_3}) = {q_3} = E | δ’(C, a) = E
δ(C, b) = {q_3, q_4, q_5}

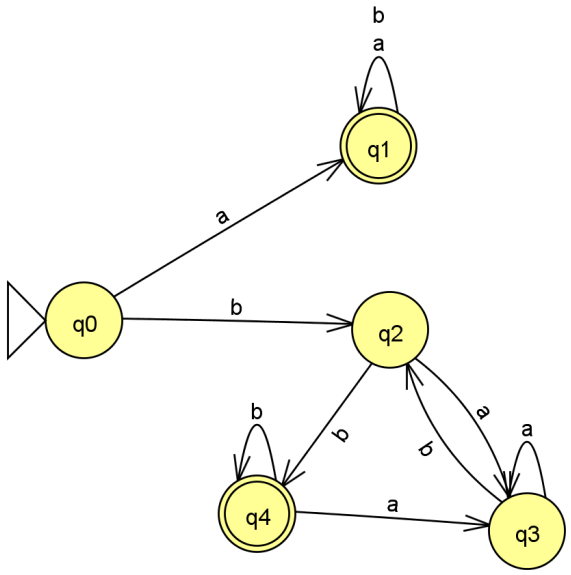
ε − clausura({q_3, q_4, q_5}) = {q_3, q_4, q_5} = F | δ'(C, b) = F
δ(D, a) = {q_2, q_3}
ε − clausura ({q_2, q_3}) = {q_2, q_3} = B | δ'(D, a) = B
δ(D, b) = {q_2, q_3, q_4, q_5}
ε − clausura ({q_2, q_3, q_4, q_5}) = {q_2, q_3, q_4, q_5} = G | δ'(D, b) = G
δ(E, a) = {q_3}
ε − clausura ({q_3}) = {q_3} = E | δ'(E, a) = E
δ(E, b) = {q_3, q_4}
ε − clausura ({q_3, q_4}) = {q_3, q_4} = C | δ'(E, b) = C
δ(F, a) = {q_3}
ε − clausura ({q_3}) = {q_3} = E | δ'(F, a) = E
δ(F, b) = {q_3, q_4, q_5}
ε − clausura ({q_3, q_4, q_5}) = {q_3, q_4, q_5} = F | δ'(F, b) = F
δ(G, a) = {q_2, q_3}
ε − clausura ({q_2, q_3}) = {q_2, q_3} = B | δ'(G, a) = B
δ(G, b) = {q_2, q_3, q_4, q_5}
ε − clausura ({q_2, q_3, q_4, q_5}) = {q_2, q_3, q_4, q_5} = G | δ'(G, b) = G

DFA resultante:



Para conseguir el DFA mínimo aplico el algoritmo de minimización de estados.

Π = {{B, D, F, G}, {A, C, E}}
Π = {{F}, {B, D, G}, {A, C, E}}
Π = {{F}, {B, D, G}, {A}. {C}, {E}}



4. **Diseñar un autómata finito no determinista que reconozca cadenas sobre el alfabeto Σ = {a, b} que empiecen por "a" y terminen en "bb". A partir del NFA diseñado, obtenga un DFA mínimo equivalente.**

Este autómata finito se puede conseguir haciendo una concatenación de los apartados 1 y 2 del ejercicio 4 y quitando un estado final.

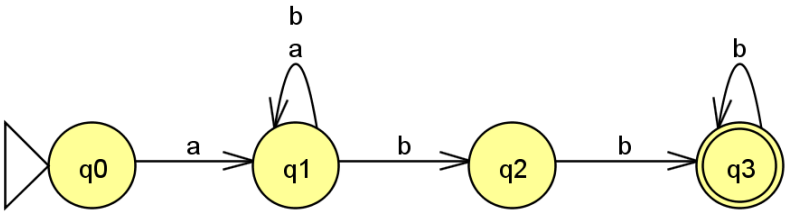


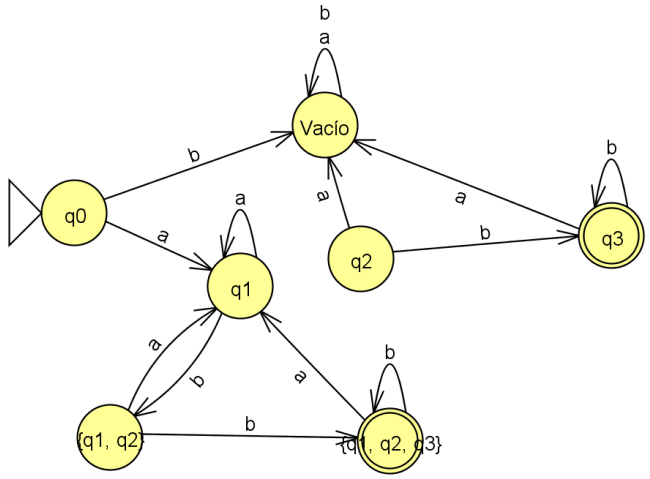
abb	Accept
abbababababbabb	Accept
aaaaaaaabb	Accept
aabbababababbabaaabb	Accept
aaabbbabb	Accept
	Reject
a	Reject
bb	Reject
abbab	Reject
bbaa	Reject

A continuación aplicaré el algoritmo de construcción de subconjuntos a un NFA sin ε − transiciones

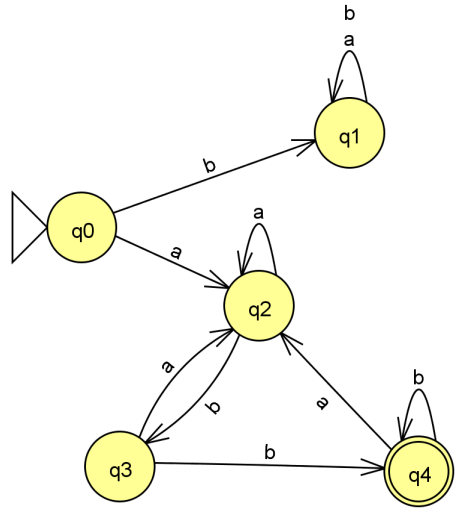
$Q' = 2Q = \{\varnothing, \{q_0\}, \{q_1\}, \{q_2\}, \{q_3\}, \{q_0, q_1\}, \{q_1, q_2\}, \{q_1, q_2, q_3\}, \ldots\}$
El resto de estados no serán necesarios ya que no son alcanzables desde el estado inicial

$\delta'(\varnothing, a) = \varnothing = \delta'(\varnothing, b)$
$\delta'(\{q_0\}, a) = \{q_1\}$
$\delta'(\{q_0\}, b) = \varnothing$
$\delta'(\{q_1\}, a) = \{q_1\}$
$\delta'(\{q_1\}, b) = \{q_1, q_2\}$
$\delta'(\{q_2\}, a) = \varnothing$
$\delta'(\{q_2\}, b) = \{q_3\}$
$\delta'(\{q_3\}, a) = \varnothing$
$\delta'(\{q_3\}, b) = \{q_3\}$
$\delta'(\{q_1, q_2\}, a) = \{q_1\} \cup \varnothing = \{q_1\}$
$\delta'(\{q_1, q_2\}, b) = \{q_1, q_2\} \cup \{q_3\} = \{q_1, q_2, q_3\}$
$\delta'(\{q_1, q_2, q_3\}, a) = \{q_1\} \cup \varnothing = \{q_1\}$
$\delta'(\{q_1, q_2, q_3\}, b) = \{q_1, q_2\} \cup \{q_3\} = \{q_1, q_2, q_3\}$

DFA resultante:



Al dibujarlo me doy cuenta que tanto q_2 como q_3 son inalcanzables desde el estado inicial así que podemos eliminarlos.



Ahora aplicaré el algoritmo de minimización de estados

$\Pi = \{\{q_4\}, \{q_0, q_1, q_2, q_3\}\}$

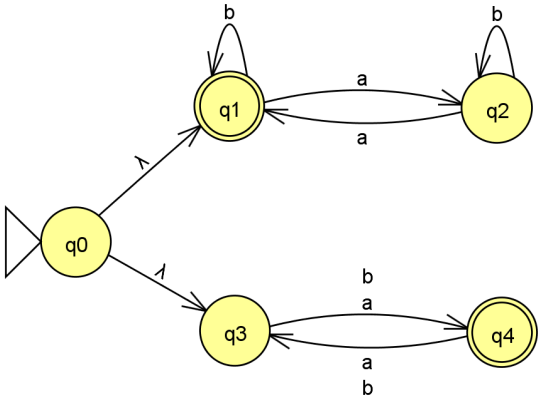
$\Pi = \{\{q_4\}, \{q_0, q_1, q_3\}, \{q_2\}\}$

$\Pi = \{\{q_4\}, \{q_0\}, \{q_3\}, \{q_1\}, \{q_2\}\}$

El autómata anterior no se podía minimizar más (ya era mínimo)

5. **Diseñar un autómata finito no determinista que reconozca cadenas sobre el alfabeto Σ = {a, b} con número de "a 's" par o longitud impar. A partir del NFA diseñado, obtenga un DFA mínimo equivalente.**

Uniendo los apartados 1 y 2 del ejercicio 3 a partir de dos ε-transiciones puedo obtener el NFA

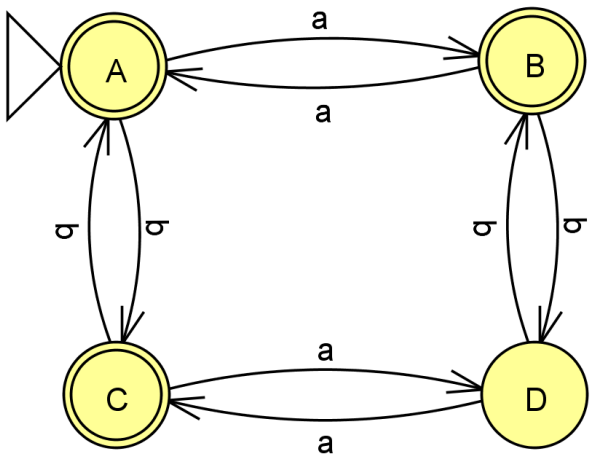


	Accept
aa	Accept
bab	Accept
aabba	Accept
bba	Accept
ab	Reject
ba	Reject
bbaaba	Reject
bbba	Reject
aaab	Reject

A continuación aplicaré el algoritmo de construcción de subconjuntos

q0’ = ε − clausura(q0) = {q0, q1, q3}
Como se trata de una disyunción a partir de ε-transiciones podemos simplificarlo diciendo que A es el estado inicial y quitar a q0 de su conjunto.
A = $\{q_1, q_3\}$
δ(A, a) = $\{q_2,q_4\}$
ε − clausura($\{q_2,q_4\}$) = $\{q_2,q_4\}$ = B | δ′(A, a) = B
δ(A, b) = $\{q_1,q_4\}$
ε − clausura($\{q_1,q_4\}$) = $\{q_1,q_4\}$ = C | δ′(A, b) = C
δ(B, a) = $\{q_1,q_3\}$
ε − clausura($\{q_1,q_3\}$) = $\{q_1, q_3\}$ = A | δ′(B, a) = A
δ(B, b) = $\{q_2,q_3\}$
ε − clausura($\{q_2,q_3\}$) = $\{q_2,q_3\}$ = D | δ′(B, b) = D
δ(C, a) = $\{q_2,q_3\}$
ε − clausura($\{q_2,q_3\}$) = $\{q_2,q_3\}$ = B | δ′(C, a) = D
δ(C, b) = $\{q_1,q_3\}$
ε − clausura($\{q_1,q_3\}$) = $\{q_1,q_3\}$ = A | δ′(C, b) = A
δ(D, a) = $\{q_1,q_4\}$
ε − clausura($\{q_1,q_4\}$) = $\{q_1,q_4\}$ = C | δ′(D, a) = C
δ(D, b) = $\{q_2,q_4\}$
ε − clausura($\{q_2,q_4\}$) = $\{q_2,q_4\}$ = B | δ′(D, b) = B

DFA resultante:

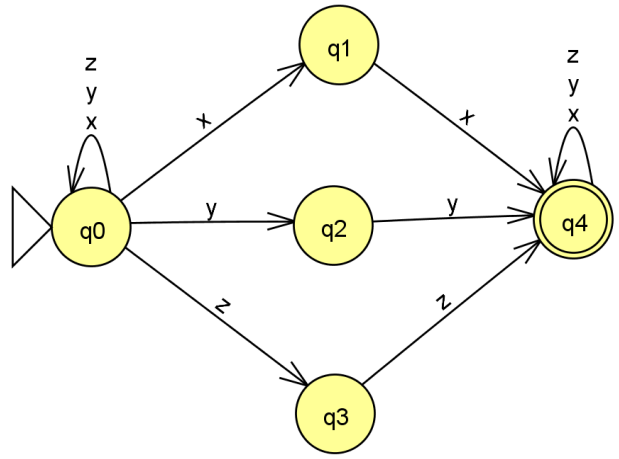


Aplicamos el algoritmo de minimización de estados
Π = {{A, B, C}, {D}}
Π = {{A}, {B, C}, {D}}
Π = {{A}, {B}, {C}, {D}}

El resultado obtenido anteriormente ya era un autómata con una cantidad mínima de estados

6. **Diseñar un autómata finito no determinista que reconozca cadenas sobre el alfabeto {x, y, z} que contengan al menos dos símbolos iguales consecutivos. A partir del NFA diseñado, obtenga un DFA mínimo equivalente.**

Son necesarios, un primer estado que acepte cualquier símbolo, otros tres estado que sirvan de ramas para comprobar que se obtienen dos símbolos iguales consecutivos y, un último estado de aceptación que acepte una vez más cualquier símbolo.



xx	Accept
yxyzyxxzz	Accept
xyyz	Accept
yzzx	Accept
yxyxyzz	Accept
	Reject
xyz	Reject
x	Reject
y	Reject
z	Reject

A continuación aplicamos el algoritmo de construcción de subconjuntos a un NFA sin ε − transiciones

$Q' = 2^Q = \{\{q_0\}, \{q_0, q_1\}, \{q_0, q_2\}, \{q_0, q_3\}, \{q_0, q_1, q_4\}, \{q_0, q_2, q_4\}, \{q_0, q_3, q_4\}, \ldots\}$

El resto de conjuntos no son necesarios ya que no son alcanzables desde el estado inicial

$\delta'(q_0, x) = \{q_0, q_1\}$
$\delta'(q_0, y) = \{q_0, q_2\}$
$\delta'(q_0, z) = \{q_0, q_3\}$

$\delta'(\{q_0, q_1\}, x) = \{q_0, q_1, q_4\}$
$\delta'(\{q_0, q_1\}, y) = \{q_0, q_2\}$
$\delta'(\{q_0, q_1\}, z) = \{q_0, q_3\}$

$\delta'(\{q_0, q_2\}, x) = \{q_0, q_1\}$
$\delta'(\{q_0, q_2\}, y) = \{q_0, q_2, q_4\}$
$\delta'(\{q_0, q_2\}, z) = \{q_0, q_3\}$

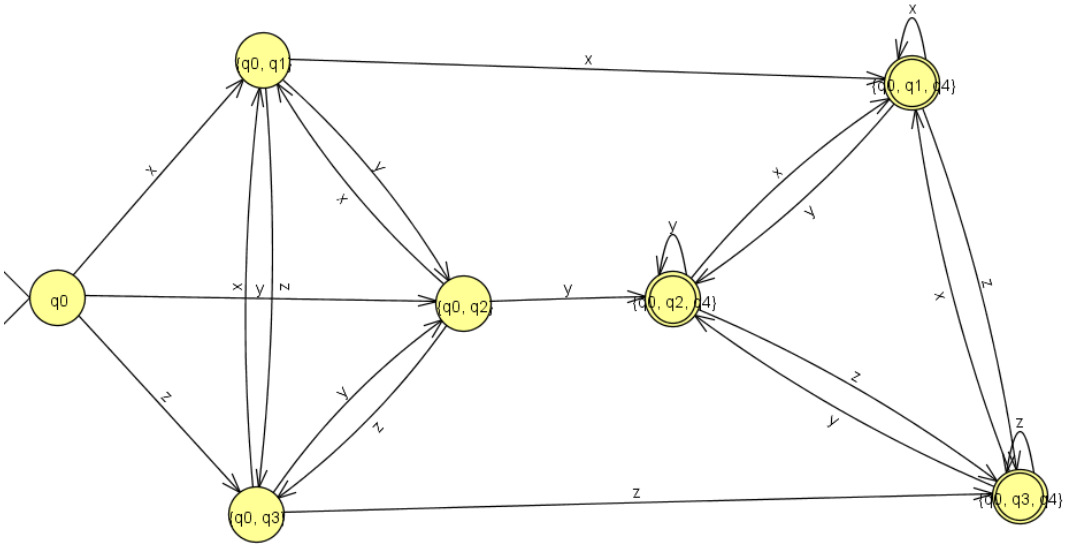
$\delta'(\{q_0, q_3\}, x) = \{q_0, q_1\}$
$\delta'(\{q_0, q_3\}, y) = \{q_0, q_2\}$
$\delta'(\{q_0, q_3\}, z) = \{q_0, q_3, q_4\}$

$\delta'(\{q_0, q_1, q_4\}, x) = \{q_0, q_1, q_4\}$
$\delta'(\{q_0, q_1, q_4\}, y) = \{q_0, q_2, q_4\}$
$\delta'(\{q_0, q_1, q_4\}, z) = \{q_0, q_3, q_4\}$

$\delta'(\{q_0, q_2, q_4\}, x) = \{q_0, q_1, q_4\}$
$\delta'(\{q_0, q_2, q_4\}, y) = \{q_0, q_2, q_4\}$
$\delta'(\{q_0, q_2, q_4\}, z) = \{q_0, q_3, q_4\}$

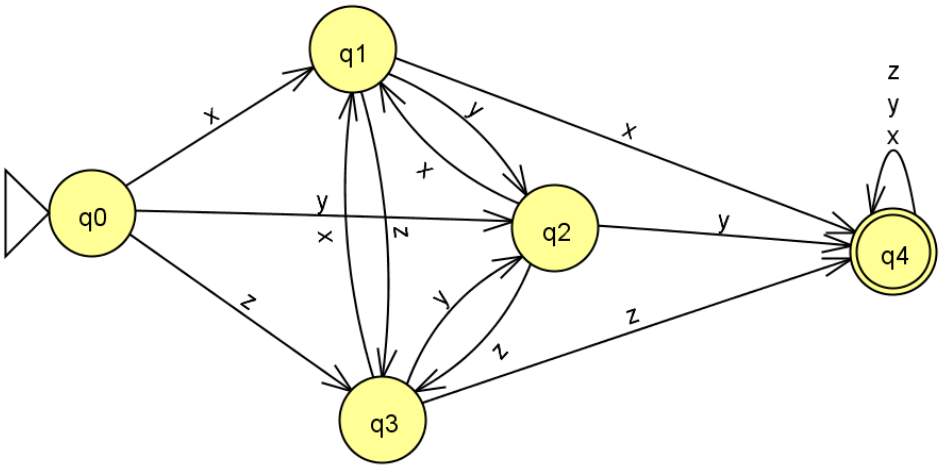
$\delta'(\{q_0, q_3, q_4\}, x) = \{q_0, q_1, q_4\}$
$\delta'(\{q_0, q_3, q_4\}, y) = \{q_0, q_2, q_4\}$
$\delta'(\{q_0, q_3, q_4\}, x) = \{q_0, q_3, q_4\}$

DFA resultante:



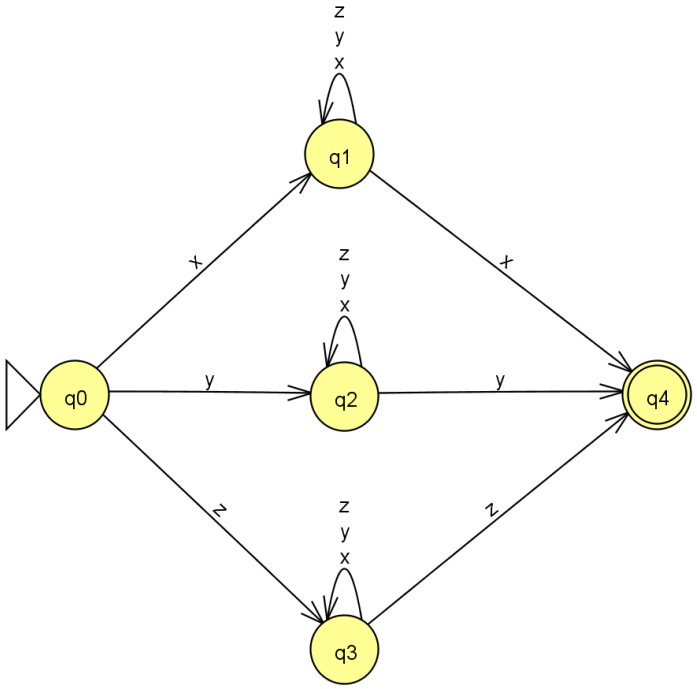
Asignando nombres distintos a los estados para un mayor entendimiento realizamos el algoritmo de minimización de estados.
$q_0 = q_0$
$q_1 = \{q_0, q_1\}$
$q_2 = \{q_0, q_2\}$
$q_3 = \{q_0, q_3\}$
$q_4 = \{q_0, q_1, q_4\}$
$q_5 = \{q_0, q_2, q_4\}$
$q_6 = \{q_0, q_3, q_4\}$

$\Pi = \{\{q_4,q_5,q_6\}, \{q_0,q_1,q_2,q_3\}\}$
$\Pi = \{\{q_4,q_5,q_6\}, \{q_0\}, \{q_1\}, \{q_2\}, \{q_3\}\}$



7. **Diseñar un autómata finito no determinista que reconozca cadenas w sobre el alfabeto {x, y, z}, con |w| ≥ 2, tales que w empieza y termina por el mismo símbolo. A partir del NFA diseñado, obtenga un DFA mínimo equivalente.**

El autómata se divide en 3 ramas que diferencian cuál fue el primer símbolo, permite cualquier cantidad de símbolos iguales o diferentes en medio y sólo llegan al estado de aceptación a través de una entrada igual a la primera.



xx	Accept
yy	Accept
zz	Accept
xyzyxyzyxx	Accept
yzzxyzxyy	Accept
	Reject
xy	Reject
yz	Reject
zx	Reject
zxyzxyyzxyzxy	Reject

A continuación aplicamos el algoritmo de construcción de subconjuntos a un NFA sin ε − transiciones

$Q' = 2Q = \{\{q_0\}, \{q_1\}, \{q_2\}, \{q_3\}, \{q_1, q_4\}, \{q_2, q_4\}, \{q_3, q_4\}, \ldots\}$
El resto de conjuntos no son necesarios ya que no son alcanzables desde el estado inicial

$\delta'(q_0, x) = \{q_1\}$
$\delta'(q_0, y) = \{q_2\}$
$\delta'(q_0, z) = \{q_3\}$

$\delta'(\{q_1\}, x) = \{q_1, q_4\}$
$\delta'(\{q_1\}, y) = \{q_1\}$
$\delta'(\{q_1\}, z) = \{q_1\}$

$\delta'(\{q_2\}, x) = \{q_2\}$
$\delta'(\{q_2\}, y) = \{q_2, q_4\}$
$\delta'(\{q_2\}, z) = \{q_2\}$

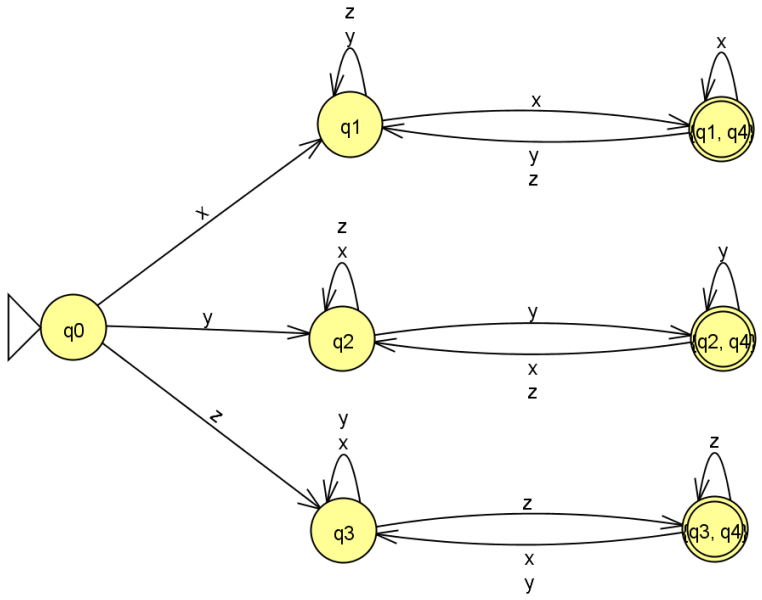
$\delta'(\{q_3\}, x) = \{q_3\}$
$\delta'(\{q_3\}, y) = \{q_3\}$
$\delta'(\{q_3\}, z) = \{q_3, q_4\}$

$\delta'(\{q_1, q_4\}, x) = \{q_1, q_4\}$
$\delta'(\{q_1, q_4\}, y) = \{q_1\}$
$\delta'(\{q_1, q_4\}, z) = \{q_1\}$

$\delta'(\{q_2, q_4\}, x) = \{q_2\}$
$\delta'(\{q_2, q_4\}, y) = \{q_2, q_4\}$
$\delta'(\{q_2, q_4\}, z) = \{q_2\}$

$\delta'(\{q_3, q_4\}, x) = \{q_3\}$
$\delta'(\{q_3, q_4\}, y) = \{q_3\}$
$\delta'(\{q_3, q_4\}, z) = \{q_3, q_4\}$

DFA resultante:



Asignando nombres distintos a los estados para un mayor entendimiento realizamos el algoritmo de minimización de estados.

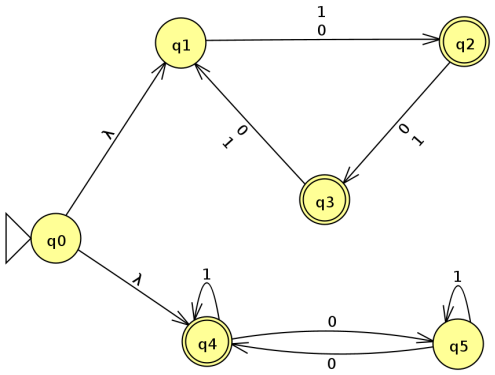
$q_0 = q_0$
$q_1 = q_1$
$q_2 = q_2$
$q_3 = q_3$
$q_4 = \{q_1, q_4\}$
$q_5 = \{q_2, q_4\}$
$q_6 = \{q_3, q_4\}$

$\Pi = \{\{q_4, q_5, q_6\}, \{q_0, q_1, q_2, q_3\}\}$
$\Pi = \{\{q_0\}, \{q_1\}, \{q_2\}, \{q_3\}, \{q_4\}, \{q_5\}, \{q_6\}\}$

El autómata finito anterior ya es mínimo,

Ejercicio propuesto

Alfabeto = {0, 1}
Obtener el DFA mínimo de las cadenas con longitud no múltiplo de 3 o nº de ceros par.
Para obtener la solución primero creo un NFA formado por la disyunción entre el DFA de las cadenas no múltiplos de 3 y el DFA de las cadenas con cantidad de 0 par. Esta disyunción la puedo realizar a partir de dos ε-transiciones.



	Accept
10	Accept
01	Accept
100	Accept
0000	Accept
110	Reject
101	Reject
111000	Reject
011	Reject
101	Reject

Algoritmo de construcción de subconjuntos

q_0' = ε − clausura(q_0) = {q_0, q_1, q_4}
A = {q_1, q_4}
δ(A, 0) = {q_2, q_5}
ε − clausura({q_2, q_5}) = {q_2, q_5} = B | δ'(A, 0) = B
δ(A, 1) = {q_2, q_4}
ε − clausura({q_2, q_4}) = {q_2, q_4} = C | δ'(A, 1) = C

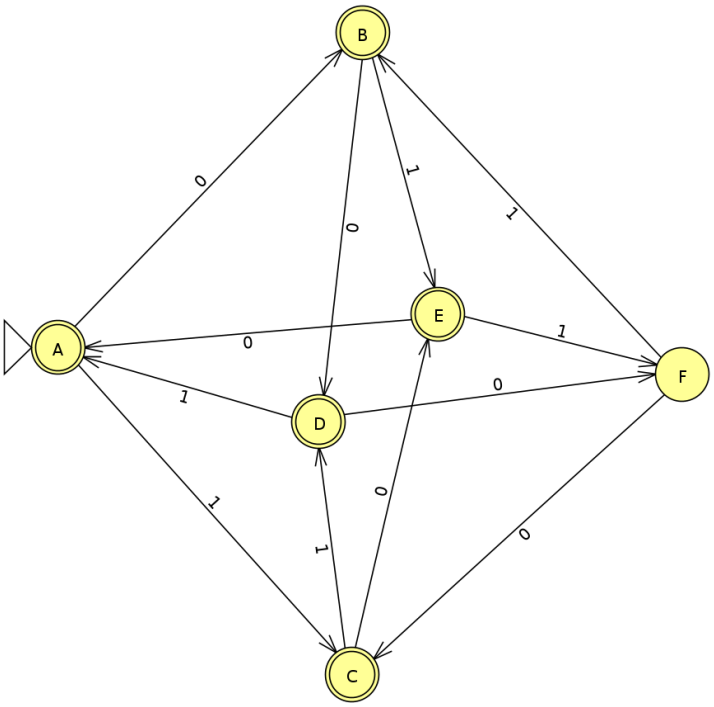
δ(B, 0) = {q_3, q_4}
ε − clausura({q_3, q_4}) = {q_3, q_4} = D | δ'(B, 0) = D
δ(B, 1) = {q_3, q_5}
ε − clausura({q_3, q_5}) = {q_3, q_5} = E | δ'(B, 1) = E

δ(C, 0) = {q_3, q_5}
ε − clausura({q_3, q_5}) = {q_3, q_5} = E | δ'(C, 0) = E
δ(C, 1) = {q_3, q_4}
ε − clausura({q_3, q_4}) = {q_3, q_4} = D | δ'(C, 1) = D

δ(D, 0) = {q_1, q_5}
ε − clausura({q_1, q_5}) = {q_1, q_5} = F | δ'(D, 0) = F
δ(D, 1) = {q_1, q_4}
ε − clausura({q_1, q_4}) = {q_1, q_4} = A | δ'(D, 1) = A

δ(E, 0) = {q_1, q_4}
ε − clausura({q_1, q_4}) = {q_1, q_4} = A | δ'(E, 0) = A
δ(E, 1) = {q_1, q_5}
ε − clausura({q_1, q_5}) = {q_1, q_5} = F | δ'(E, 1) = F

δ(F, 0) = {q_2, q_4}
ε − clausura({q_2, q_4}) = {q_2, q_4} = C | δ'(F, 0) = C
δ(F, 1) = {q_2, q_5}
ε − clausura({q_2, q_5}) = {q_2, q_5} = B | δ'(F, 1) = B



Algoritmo de minimización de estados

Π = {{A, B, C, D, E}, {F}}

Π = {{A}, {B, C, D, E}, {F}}

Π = {{A}, {B}, {C, D, E}, {F}}

Π = {{A}, {B}, {C}, {D, E}, {F}}

Π = {{A}, {B}, {C}, {D}, {E}, {F}}

El autómata anterior ya tenía una cantidad mínima de estados